

Embedded C Programs and Output Screenshots

1. Temperature Sensor using ADC and Cooling Fan Control

```
#include <stdio.h>

#define TEMP_THRESHOLD 30

int ADC_Read()
{
    int adcValue;
    printf("Enter ADC value (0-1023): ");
    scanf("%d", &adcValue);
    return adcValue;
}

int GetTemperature(int adcValue)
{
    return (adcValue * 500) / 1023;
}

void ControlFan(int temperature)
{
    if (temperature >= TEMP_THRESHOLD)
        printf("Cooling Fan: ON\n");
    else
        printf("Cooling Fan: OFF\n");
}

int main()
{
    int adcValue = ADC_Read();
    int temperature = GetTemperature(adcValue);

    printf("Temperature = %d °C\n", temperature);
    ControlFan(temperature);

    return 0;
}
```

2. Standard C vs Embedded C Comparison

```
#include <stdio.h>
#include <time.h>

volatile unsigned char PORTA = 0x00;

void standardC_LED(int state)
{
    if(state) PORTA = 1;
    else PORTA = 0;
}

void embeddedC_LED_ON(){ PORTA |= (1<<0); }
void embeddedC_LED_OFF(){ PORTA &= ~(1<<0); }
void embeddedC_LED_TOGGLE(){ PORTA ^= (1<<0); }

int main()
{
    int i;
    clock_t start,end;
    double standardTime,embeddedTime;

    start=clock();
    for(i=0;i<1000000;i++){
        standardC_LED(1);
        standardC_LED(0);
    }
    end=clock();
    standardTime=(double)(end-start)/CLOCKS_PER_SEC;

    start=clock();
```

```

    for(i=0;i<1000000;i++){
        embeddedC_LED_ON();
        embeddedC_LED_OFF();
        embeddedC_LED_TOGGLE();
    }
    end=clock();
    embeddedTime=(double)(end-start)/CLOCKS_PER_SEC;

    printf("PORTA=%u\n",PORTA);
    printf("Standard C Time=%f\n",standardTime);
    printf("Embedded C Time=%f\n",embeddedTime);
    return 0;
}

```

Output Screenshots

The screenshot displays the Programiz C Online Compiler interface. The left sidebar contains icons for various programming languages, with C selected. The main editor area shows a C program in a file named `main.c`. The code includes headers, defines a temperature threshold, and implements functions for reading ADC values, converting them to temperature, and controlling a cooling fan. The `main` function calls these functions and prints the results. The right sidebar shows the output of the program, which includes the prompt for an ADC value, the calculated temperature, and the fan status. A message at the bottom of the output area indicates that the code execution was successful.

```

main.c
1  #include <stdio.h>
2
3  #define TEMP_THRESHOLD 30
4
5  // Function to simulate ADC reading
6  int ADC_Read()
7  {
8      int adcValue;
9      printf("Enter ADC value (0-1023): ");
10     scanf("%d", &adcValue);
11
12     return adcValue;
13 }
14
15 // Function to convert ADC value to temperature (LM35)
16 int GetTemperature(int adcValue)
17 {
18     return (adcValue * 500) / 1023;
19 }
20
21 // Function to control cooling fan
22 void ControlFan(int temperature)
23 {
24     if (temperature >= TEMP_THRESHOLD)
25         printf("Cooling Fan: ON\n");
26     else

```

Output

```

Enter ADC value (0-1023): 80
Temperature = 39 °C
Cooling Fan: ON

=== Code Execution Successful ===

```

← → ↺ programiz.com/c-programming/online-compiler/

Programiz

C Online Compiler

main.c

🔍

⚙️

🔗 Share

🚀 Run

37 printf("==== Standard C vs Embedded C ====\\n\\n");

38

39 // Standard C Performance

40 start = clock();

41 for(i = 0; i < 1000000; i++)

42 {

43 standardC_LED(1);

44 standardC_LED(0);

45 }

46 end = clock();

47 standardTime = (double)(end - start) / CLOCKS_PER_SEC;

48

49 // Embedded C Performance

50 start = clock();

51 for(i = 0; i < 1000000; i++)

52 {

53 embeddedC_LED_ON();

54 embeddedC_LED_OFF();

55 embeddedC_LED_TOGGLE();

56 }

57 end = clock();

58 embeddedTime = (double)(end - start) / CLOCKS_PER_SEC;

59

60 printf("Final Register Value (PORTA) = %u\\n\\n", PORTA);

61

62 printf("Performance Comparison\\n");

Output

Clear

==== Standard C vs Embedded C ====

Final Register Value (PORTA) = 1

Performance Comparison

Standard C Time : 0.005145 seconds

Embedded C Time : 0.007100 seconds

Analysis:

1. Standard C uses normal variables and functions.

2. Embedded C directly manipulates hardware registers.

3. Bitwise operations (&, ^, <<) are faster and memory efficient.

4. Embedded C provides better execution speed for hardware control.

5. Register-level programming reduces overhead and improves performance.

=== Code Execution Successful ===